
实验 2：数据定义与数据操纵（实验报告）

(2026 春季，智能软件与工程学院)

学号： 241880540 姓名： 汪一航

(说明：实验报告请使用 PDF 格式提交，截止日期：2026.05.12)

实验环境

[如果你不是使用本课程建议的 MySQL 数据库，请用一句话介绍你使用的操作系统、数据库管理系统、软件版本]

使用 MySQL 数据库，软件使用 DataGrip 2025.3.5

实验过程

[所有要求提交的 SQL 语句文本和执行结果截图]

第一部分：数据库创建

```
use mysql;
```

```
drop schema if exists myschool26; # 删除原有的数据模式
```

```
create schema myschool26 charset gb2312 collate gb2312_bin;
```

```
use myschool26;# 设置当前工作模式
```

```
#创建学生表 students
```

```
CREATE TABLE students
```

```
(
```

```
    sno          CHAR(9) PRIMARY KEY,
```

```
    sname        CHAR(20) NOT NULL,
```

```
    ssex         CHAR(2)  NOT NULL CHECK (ssex IN ('男', '女')),
```

```
    birthday     DATE     NOT NULL,
```

```
    dept         CHAR(20),
```

```
    enrolldate DATE
```

```
);
```

```
#创建教师表 teachers
```

```
CREATE TABLE teachers
```

```
(
```

```
    tno  CHAR(9) PRIMARY KEY,
```

```
    tname CHAR(20) NOT NULL,
```

```
    dept CHAR(20)
```

);

#创建课程表 courses

CREATE TABLE courses

```
(  
    cno    CHAR(8) PRIMARY KEY,  
    cname  CHAR(40) NOT NULL,  
    credit INT      NOT NULL CHECK (credit > 0 AND credit < 10),  
    chours INT      NOT NULL,  
    dept   CHAR(20)  
);
```

#创建课程班表 courseclass

CREATE TABLE courseclass

```
(  
    clsno  CHAR(8) PRIMARY KEY,  
    cno    CHAR(8) NOT NULL,  
    tno    CHAR(9),  
    clsyear INT      NOT NULL,  
    clsterm CHAR(4) NOT NULL CHECK (clsterm IN ('春季', '秋季', '暑期')),  
    FOREIGN KEY (cno) REFERENCES courses (cno) ON UPDATE CASCADE ON  
DELETE RESTRICT,  
    FOREIGN KEY (tno) REFERENCES teachers (tno) ON UPDATE CASCADE ON  
DELETE SET NULL  
);
```

#创建选课表 enrollment

CREATE TABLE enrollment

```
(  
    sno    CHAR(9) NOT NULL,  
    clsno  CHAR(8) NOT NULL,  
    grade  INT CHECK (grade >= 0 AND grade <= 100),  
    PRIMARY KEY (sno, clsno),  
    FOREIGN KEY (sno) REFERENCES students (sno) ON UPDATE CASCADE ON  
DELETE RESTRICT,  
    FOREIGN KEY (clsno) REFERENCES courseclass (clsno) ON UPDATE  
CASCADE ON DELETE RESTRICT  
);
```

第二部分：初始数据加载（不需要提交）

第三部分：数据查询

#1、查询满足下述条件的课程的课程号、课程名、开课院系：课程名中含有汉字‘数’；结果按照开课院系和课程号的升序排列。

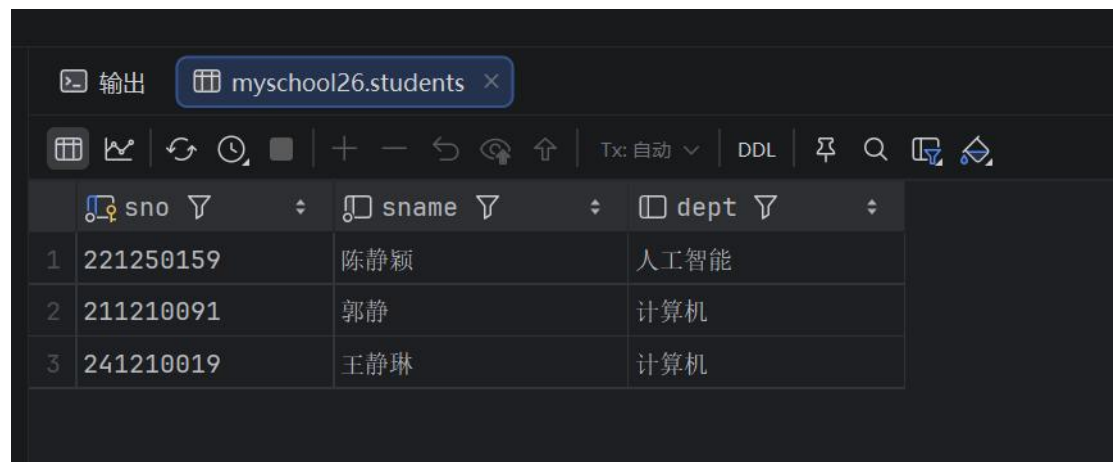
```
SELECT cno, cname, dept FROM courses WHERE cname LIKE '%数%' ORDER BY dept, cno;
```



	cno	cname	dept
1	1201	高等数学(I)	计算机
2	2205	数据结构	计算机
3	3208	数据库概论	计算机
4	3308	数据库技术	人工智能
5	1101	高等数学	数学
6	2105	数据结构	数学
7	2106	数学分析	数学

#2、查询满足下述条件的学生的学号、姓名、就读院系：姓名中的第二个汉字是‘静’；结果按照学生姓名的升序排列。

```
SELECT sno, sname, dept FROM students WHERE sname LIKE '_静%' ORDER BY sname;
```



	sno	sname	dept
1	221250159	陈静颖	人工智能
2	211210091	郭静	计算机
3	241210019	王静琳	计算机

#3.查询成绩为空值的选课记录，结果返回选课学生的学号和姓名、课程的课程号和课程名；结果按照课程号和学号的升序排列。

```
SELECT students.sno, students.sname, courses.cno, courses.cname
FROM enrollment
JOIN students ON enrollment.sno = students.sno
JOIN courseclass ON enrollment.clsno = courseclass.clsno
JOIN courses ON courseclass.cno = courses.cno
```

WHERE enrollment.grade IS NULL
ORDER BY courses.cno , students.sno ;

	sno	sname	cno	cname
1	231250073	王东阳	2105	数据结构
2	221250212	陈凯歌	3208	数据库概论

#4.查询下列学生的课程修读情况（不考虑没有成绩的课程）：2023 级、且所有成绩都及格；结果返回满足条件学生的学号，姓名，修读课程的门数、学分总数及平均成绩；并按照学号的升序输出查询结果。

```
SELECT
    s.sno AS 学号,
    s.sname AS 姓名,
    COUNT(c.cno) AS 修读门数,
    SUM(c.credit) AS 学分总数,
    AVG(e.grade) AS 平均成绩
FROM students s
    JOIN enrollment e ON s.sno = e.sno
    JOIN courseclass cc ON e.clsno = cc.clsno
    JOIN courses c ON cc.cno = c.cno
WHERE e.grade IS NOT NULL
AND YEAR(s.enrolldate) = 2023
GROUP BY s.sno
HAVING MIN(e.grade) >= 60
ORDER BY s.sno;
```

	学号	姓名	修读门数	学分总数	平均成绩
1	231220042	李语诚	3	11	83.0000
2	231250006	吴晓静	4	17	79.5000
3	231250013	吴川江	3	11	81.0000
4	231250020	吴雅婷	3	11	77.3333
5	231250021	吴迪	3	12	90.6667
6	231250045	蒋权友	4	17	86.2500
7	231250054	郑韬	3	11	91.6667
8	231250064	李依娜	3	12	79.0000
9	231250073	王东阳	3	13	83.3333

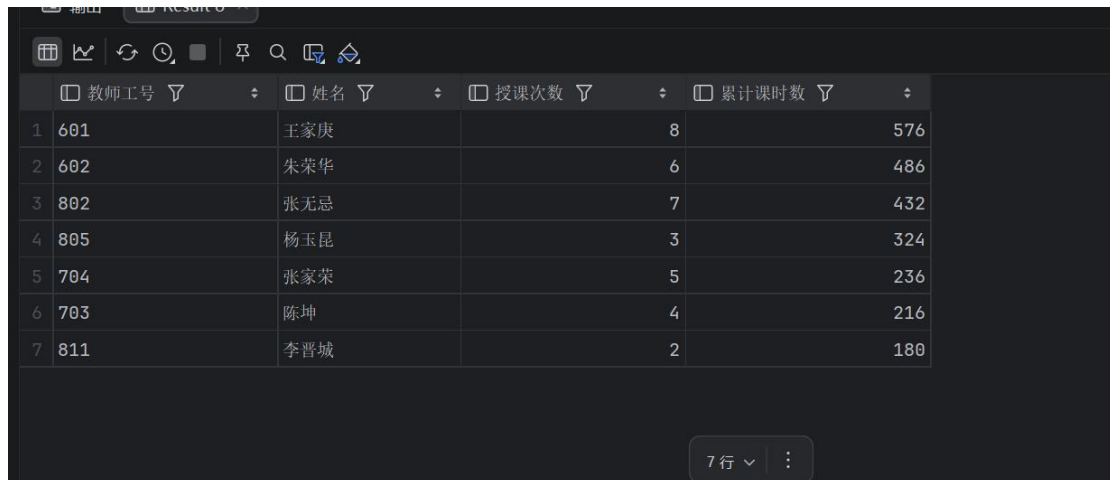
#5.查询承担过授课任务的教师的授课情况，结果返回教师的工号和姓名、授课次数（讲授的课程班数）、累计课时数。结果按照累计课时数的降序和教师工号的升序排列。

SELECT

```

t.tno AS 教师工号,
t.tname AS 姓名,
COUNT(cc.clsno) AS 授课次数,
SUM(c.chours) AS 累计课时数
FROM teachers t
      JOIN courseclass cc ON t.tno = cc.tno
      JOIN courses c ON cc.cno = c.cno
GROUP BY t.tno
ORDER BY 累计课时数 DESC, t.tno;

```



The screenshot shows a database query result with 7 rows. The columns are: 教师工号 (Teacher ID), 姓名 (Name), 授课次数 (Number of classes taught), and 累计课时数 (Total hours). The results are sorted by total hours in descending order.

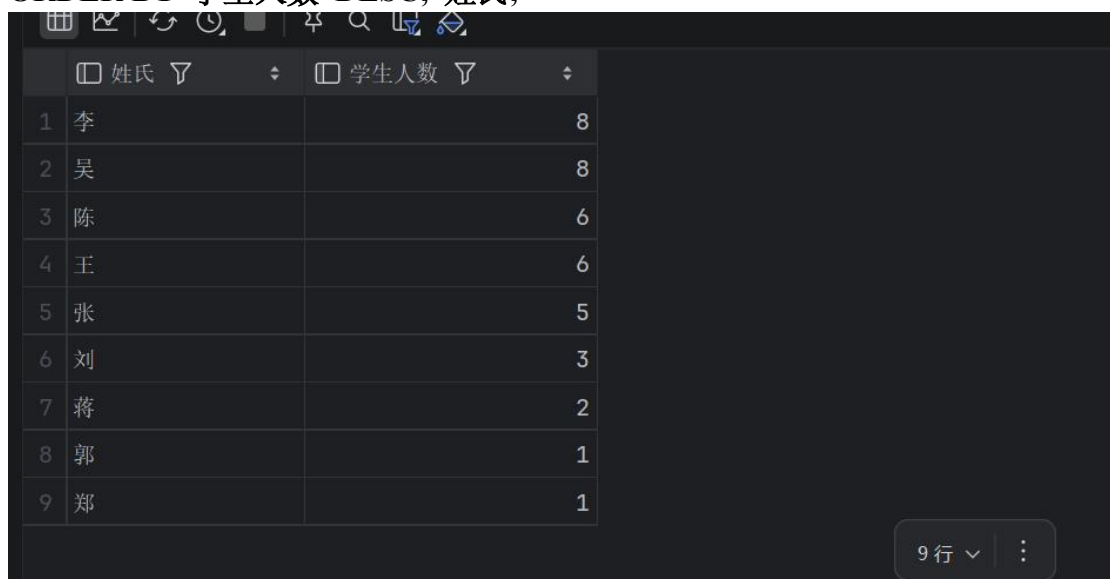
	教师工号	姓名	授课次数	累计课时数
1	601	王家庚	8	576
2	602	朱荣华	6	486
3	802	张无忌	7	432
4	805	杨玉昆	3	324
5	704	张家荣	5	236
6	703	陈坤	4	216
7	811	李晋城	2	180

#6.按照姓氏统计所有在校学生的人数，结果返回姓氏和学生人数；结果按照人数的降序和姓氏的升序排列。说明：所有学生均为单字的姓氏，不考虑复姓情况；需要使用系统内置的字符处理函数。

```

SELECT
    LEFT(sname, 1) AS 姓氏,
    COUNT(sno) AS 学生人数
FROM students
GROUP BY LEFT(sname, 1)
ORDER BY 学生人数 DESC, 姓氏;

```



The screenshot shows a database query result with 9 rows. The columns are: 姓氏 (Surname) and 学生人数 (Number of students). The results are sorted by the number of students in descending order, and then by surname in ascending order.

	姓氏	学生人数
1	李	8
2	吴	8
3	陈	6
4	王	6
5	张	5
6	刘	3
7	蒋	2
8	郭	1
9	郑	1

#7.满足下述条件的学生的就读院系、学号和姓名：2021 级、且存在自己就读学院开设的课程没有通过（没有选修、或没有成绩、或成绩不及格），结果按照学生就读院系和学号的升序排列。

```
SELECT s.dept, s.sno, s.sname
FROM students s
WHERE YEAR(s.enrolldate) = 2021
      AND (SELECT COUNT(DISTINCT cc.cno)
            FROM enrollment e
              JOIN courseclass cc ON e.clsno = cc.clsno
              JOIN courses c ON cc.cno = c.cno
            WHERE e.sno = s.sno
              AND c.dept = s.dept
              AND e.grade >= 60) < (SELECT COUNT(*)
                                     FROM courses c2
                                     WHERE c2.dept = s.dept)
ORDER BY s.dept, s.sno ;
```

	dept	sno	sname
1	计算机	211210043	刘家君
2	计算机	211210166	蒋超
3	数学	211210088	吴小芳

#8.满足下述条件的学生的就读院系、学号和姓名：2021 级、且选修通过了自己就读学院开设的所有课程，结果按照学生就读院系和学号的升序排列。

```
SELECT s.dept, s.sno, s.sname
FROM students s
WHERE YEAR(s.enrolldate) = 2021
      AND (
        SELECT COUNT(DISTINCT cc.cno)
        FROM enrollment e
          JOIN courseclass cc ON e.clsno = cc.clsno
          JOIN courses c ON cc.cno = c.cno
        WHERE e.sno = s.sno
          AND c.dept = s.dept
          AND e.grade >= 60
      ) = (
        SELECT COUNT(*)
        FROM courses c2
        WHERE c2.dept = s.dept
      )
ORDER BY s.dept, s.sno;
```

	dept	sno	sname
1	计算机	211210001	陈圣威
2	计算机	211210091	郭静
3	计算机	211210119	王伟
4	人工智能	211210042	刘菲
5	人工智能	211210112	李杰
6	数学	211210155	李程锦

#9.查询每一位教师的授课起止年份，结果返回教师的工号、姓名、工作院系、第一次授课的年份、最后一次授课的年份；如果某位老师没有承担过授课任务，则授课起止年份返回空值；结果按照工作院系和教师工号的升序排列。

```

SELECT t.tno          AS 教师工号,
       t.tname        AS 姓名,
       t.dept         AS 工作院系,
       MIN(cc.clsyear) AS 第一次授课年份,
       MAX(cc.clsyear) AS 最后一次授课年份
FROM teachers t
      LEFT JOIN courseclass cc ON t.tno = cc.tno
GROUP BY t.tno, t.tname, t.dept
ORDER BY t.dept, t.tno;

```

	教师工号	姓名	工作院系	第一次授课年份	最后一次授课年份
1	602	朱荣华	计算机	2022	2025
2	704	张家荣	计算机	2023	2025
3	777	李云龙	计算机	<null>	<null>
4	811	李晋城	计算机	2023	2024
5	703	陈坤	人工智能	2021	2024
6	802	张无忌	人工智能	2021	2024
7	601	王家庚	数学	2021	2025
8	805	杨玉昆	数学	2022	2024

#10.在一门课程的各个课程班中，查询各课程班的班平均成绩的最高值；结果返回课程的课程号、课程名、班平均成绩最高的课程班的代码、授课年份和授课学期；并按照课程号的升序和授课年份的降序输出查询结果。（在计算班平均成绩时，不需要考虑成绩为空值的选课元组）

```

SELECT c.cno          AS 课程号,
       c.cname        AS 课程名,
       cc.clsno       AS 课程班代码,
       cc.clsyear     AS 授课年份,
       cc.clsterm     AS 授课学期
FROM courses c
      JOIN courseclass cc ON c.cno = cc.cno
      JOIN enrollment e ON cc.clsno = e.clsno
WHERE e.grade IS NOT NULL
GROUP BY cc.clsno, c.cno, cc.clsyear

```

```

HAVING AVG(e.grade) >= ALL (SELECT AVG(e2.grade)
                             FROM courseclass cc2
                             JOIN enrollment e2 ON cc2.clsno = e2.clsno
                             WHERE e2.grade IS NOT NULL
                             AND cc2.cno = c.cno
                             GROUP BY cc2.clsno)
ORDER BY c.cno, cc.clsyear DESC;

```

	课程号	课程名	课程班代码	授课年份	授课学期
1	1101	高等数学	2411101	2024	秋季
2	1201	高等数学(I)	2221201	2023	春季
3	1202	人工智能导论	2311202	2023	秋季
4	1302	人工智能导论	2111302	2021	秋季
5	2105	数据结构	2422105	2025	春季
6	2106	数学分析	2312106	2023	秋季
7	2205	数据结构	2212205	2022	秋季
8	2305	机器学习	2212305	2022	秋季
9	3207	操作系统	2313207	2023	秋季
10	3208	数据库概论	2423208	2025	春季
11	3308	数据库技术	2323308	2024	春季

第四部分：数据更新

#1.关闭事务自动提交标志：set autocommit=0;
set autocommit=0;

```

[2026-04-29 16:41:07] 已连接
[2026-04-29 16:41:07] myschool26> use myschool26
[2026-04-29 16:41:07] 在 6 ms 内完成
[2026-04-29 16:41:07] myschool26> set autocommit=0
[2026-04-29 16:41:07] 在 6 ms 内完成

```

#2.在课程班表 courseclass 中插入一条元组：2025 年秋季，课程号 3208，授课教师工号 704，课程班号 2513208

```

INSERT INTO courseclass (clsno, cno, tno, clsyear, clsterm)
VALUES ('2513208', '3208', '704', 2025, '秋季');

```

```

[2026-04-29 16:43:43] myschool26> INSERT INTO courseclass (clsno, cno, tno, clsyear, clsterm)
VALUES ('2513208', '3208', '704', 2025, '秋季')
[2026-04-29 16:43:43] 7 ms 中有 1 行受到影响

```

#3.在选课表 enrollment 中插入如下的选课元组：计算机学院 2024 级的所有同学关于课程班（课程班号 2513208）的选课元组（无课程成绩）

```

INSERT INTO enrollment (sno, clsno, grade)
SELECT

```



```

sno,
'2513208' AS clsno,
NULL AS grade
FROM students
WHERE dept = '计算机'
AND YEAR(enrolldate) = 2024;

```

```

[2026-04-29 16:54:26] myschool26> INSERT INTO enrollment (sno, clsno, grade)
SELECT
    sno,
    '2513208' AS clsno,
    NULL AS grade
FROM students
WHERE dept = '计算机'
AND YEAR(enrolldate) = 2024
[2026-04-29 16:54:26] 4 ms 中有 5 行受到影响

```

#4.使用 commit 命令提交当前事务的执行结果。
commit;

```

[2026-04-29 17:03:17] myschool26> commit
[2026-04-29 17:03:17] 在 8 ms 内完成

```

#5.查询成绩为空值的选课元组，结果返回：学生的学号和就读院系、课程的课程名和开课院系、授课教师的姓名、课程班的授课年份和学期。结果按照学号和课程号的升序排列。

```

SELECT
    s.sno AS 学号,
    s.dept AS 就读院系,
    c.cname AS 课程名,
    c.dept AS 开课院系,
    t.tname AS 授课教师,
    cc.clsyear AS 授课年份,
    cc.clsterm AS 授课学期
FROM enrollment e
    JOIN students s ON e.sno = s.sno
    JOIN courseclass cc ON e.clsno = cc.clsno
    JOIN courses c ON cc.cno = c.cno
    JOIN teachers t ON cc.tno = t.tno
WHERE e.grade IS NULL
ORDER BY s.sno, c.cno;

```

	学号 ▾	就读院系 ▾	课程名 ▾	开课院系 ▾	授课教师 ▾	授课年份 ▾	授课学期 ▾
1	221250212	计算机	数据库概论	计算机	张家荣	2025	春季
2	231250073	数学	数据结构	数学	朱荣华	2025	春季
3	241210012	计算机	数据库概论	计算机	张家荣	2025	秋季
4	241210019	计算机	数据库概论	计算机	张家荣	2025	秋季
5	241210023	计算机	数据库概论	计算机	张家荣	2025	秋季
6	241210047	计算机	数据库概论	计算机	张家荣	2025	秋季
7	241210123	计算机	数据库概论	计算机	张家荣	2025	秋季

#1.关闭事务自动提交标志: set autocommit=0;

```
set autocommit=0;
```

```
[2026-04-29 17:08:38] myschool26> set autocommit=0
[2026-04-29 17:08:38] 在 1 ms 内完成
```

#2.将下述选课元组上的成绩值修改为 68: 学号 211210166, 课程班号 2111202

```
UPDATE enrollment
```

```
SET grade = 68
```

```
WHERE sno = '211210166'
```

```
AND clsno = '2111202';
```

```
[2026-04-29 17:10:05] myschool26> UPDATE enrollment
                                SET grade = 68
                                WHERE sno = '211210166'
                                AND clsno = '2111202'
[2026-04-29 17:10:05] 3 ms 中有 1 行受到影响
```

#3.查询满足下述条件的学生的学号、姓名和就读院系: 2021 级、且选修通过了‘计算机’学院开设的所有课程, 结果按照学生学号的升序排列。

```
SELECT
```

```
    s.sno AS 学号,
```

```
    s.sname AS 姓名,
```

```
    s.dept AS 就读院系
```

```
FROM students s
```

```
WHERE YEAR(s.enrolldate) = 2021
```

```
AND (
```

```
    SELECT COUNT(DISTINCT cc.cno)
```

```
    FROM enrollment e
```

```
        JOIN courseclass cc ON e.clsno = cc.clsno
```

```
        JOIN courses c ON cc.cno = c.cno
```

```
    WHERE e.sno = s.sno
```

```
        AND c.dept = '计算机'
```

```
        AND e.grade >= 60
```

```
) = (
```

```
    SELECT COUNT(*)
```

```
    FROM courses c2
```

```
    WHERE c2.dept = '计算机'
```

```
)
```

```
ORDER BY s.sno;
```

	□ 学号 ▾	□ 姓名 ▾	□ 就读院系 ▾
1	211210001	陈圣威	计算机
2	211210091	郭静	计算机
3	211210119	王伟	计算机
4	211210166	蒋超	计算机

#4.使用 rollback 命令放弃当前事务的更新结果，结束当前事务。

rollback;

```
[2026-04-29 17:16:16] myschool26> rollback
[2026-04-29 17:16:16] 在 5 ms 内完成
```

#5.将下列选课元组上的成绩修改为空值：跨院系的课程选修（学生的就读院系和课程的开课院系不一样）

UPDATE enrollment e

JOIN students s ON e.sno = s.sno

JOIN courseclass cc ON e.clsno = cc.clsno

JOIN courses c ON cc.cno = c.cno

SET e.grade = NULL

WHERE s.dept != c.dept;

```
[2026-04-29 17:18:52] myschool26> UPDATE enrollment e
      JOIN students s ON e.sno = s.sno
      JOIN courseclass cc ON e.clsno = cc.clsno
      JOIN courses c ON cc.cno = c.cno
      SET e.grade = NULL
      WHERE s.dept != c.dept
[2026-04-29 17:18:52] 9 ms 中有 23 行受到影响
```

#6.使用 rollback 命令放弃当前事务的更新结果，结束当前事务。

rollback;

```
[2026-04-29 17:19:47] myschool26> rollback
[2026-04-29 17:19:47] 在 7 ms 内完成
```

#1.关闭事务自动提交标志：set autocommit=0;

set autocommit=0;

```
[2026-04-29 17:20:53] myschool26> set autocommit=0
[2026-04-29 17:20:53] 在 2 ms 内完成
```

#2.删除‘计算机’学院开设的‘人工智能导论’这门课程及其所有的选课元组。

DELETE e FROM enrollment e

JOIN courseclass cc ON e.clsno = cc.clsno

```
JOIN courses c ON cc.cno = c.cno
WHERE c.dept = '计算机' AND c.cname = '人工智能导论';
```

```
DELETE cc FROM courseclass cc
JOIN courses c ON cc.cno = c.cno
WHERE c.dept = '计算机' AND c.cname = '人工智能导论';
```

```
DELETE FROM courses
WHERE dept = '计算机' AND cname = '人工智能导论';
```

```
[2026-04-29 17:36:41] myschool26> DELETE e FROM enrollment e
                                JOIN courseclass cc ON e.clsno = cc.clsno
                                JOIN courses c ON cc.cno = c.cno
                                WHERE c.dept = '计算机' AND c.cname = '人工智能导论'
[2026-04-29 17:36:41] 6 ms 中有 31 行受到影响
[2026-04-29 17:36:41] myschool26> DELETE cc FROM courseclass cc
                                JOIN courses c ON cc.cno = c.cno
                                WHERE c.dept = '计算机' AND c.cname = '人工智能导论'
[2026-04-29 17:36:41] 3 ms 中有 4 行受到影响
[2026-04-29 17:36:41] myschool26> DELETE FROM courses
                                WHERE dept = '计算机' AND cname = '人工智能导论'
[2026-04-29 17:36:41] 2 ms 中有 1 行受到影响
```

#3.使用 rollback 命令放弃当前事务的更新结果，结束当前事务。

```
rollback;
```

```
[2026-04-29 17:38:04] myschool26> rollback
[2026-04-29 17:38:04] 在 11 ms 内完成
```

#4.删除课程班号为 2513208 的课程班元组及其所有的选课元组。

```
DELETE FROM enrollment
WHERE clsno = '2513208';
```

```
DELETE FROM courseclass
WHERE clsno = '2513208';
```

```
[2026-04-29 17:39:41] myschool26> DELETE FROM enrollment
                                WHERE clsno = '2513208'
[2026-04-29 17:39:41] 2 ms 中有 5 行受到影响
[2026-04-29 17:39:41] myschool26> DELETE FROM courseclass
                                WHERE clsno = '2513208'
[2026-04-29 17:39:41] 2 ms 中有 1 行受到影响
```

#5.使用 commit 命令提交当前事务的执行结果。

```
commit;
```

```
[2026-04-29 17:40:39] myschool26> commit
[2026-04-29 17:40:39] 在 5 ms 内完成
```

第五部分：视图创建与访问

#1.创建第一个视图：用于查询每一门课程的累计修读学生人数和该门课程的平均成绩（视图属性包括：课程号、累计修读学生人数、平均成绩）

```
CREATE VIEW v_course_statistics AS
SELECT
    cc.cno AS 课程号,
    COUNT(e.sno) AS 累计修读学生人数,
    AVG(e.grade) AS 平均成绩
FROM courseclass cc
    JOIN enrollment e ON cc.clsno = e.clsno
GROUP BY cc.cno;
```

```
[2026-04-30 10:21:42] myschool26> use myschool26
[2026-04-30 10:21:42] 在 9 ms 内完成
[2026-04-30 10:21:42] myschool26> /* 第五部分：视图创建与访问 */
#1.创建第一个视图：用于查询每一门课程的累计修读学生人数和该门课程的平均成绩（视图属性包括：课程号、累计修读学生人数、
CREATE VIEW v_course_statistics AS
SELECT
    cc.cno AS 课程号,
    COUNT(e.sno) AS 累计修读学生人数,
    AVG(e.grade) AS 平均成绩
FROM courseclass cc
    JOIN enrollment e ON cc.clsno = e.clsno
GROUP BY cc.cno
[2026-04-30 10:21:42] 在 32 ms 内完成
```

#2.视图的嵌套定义和查询

#(1)创建第二个视图，用于查询每一门课程的累计修读学生人数、成绩大于课程平均成绩的学生人数（视图属性包括：课程号、累计修读学生人数、大于课程平均成绩的学生人数）

```
CREATE VIEW v_course_performance AS
SELECT
    v1.课程号,
    v1.累计修读学生人数,
    SUM(CASE WHEN e.grade > v1.平均成绩 THEN 1 ELSE 0 END) AS 大于课程平均
    成绩的学生人数
FROM v_course_statistics v1
    JOIN courseclass cc ON v1.课程号 = cc.cno
    JOIN enrollment e ON cc.clsno = e.clsno
GROUP BY v1.课程号, v1.累计修读学生人数;
```

```
[2026-04-30 10:25:47] myschool26> #2.视图的嵌套定义和查询
#(1)创建第二个视图，用于查询每一门课程的累计修读学生人数、成绩大于课程平均成绩的学生人数（视图属性包括：课程号、累计修
CREATE VIEW v_course_performance AS
SELECT
    v1.课程号,
    v1.累计修读学生人数,
    -- 条件聚合 (Conditional Aggregation)：统计成绩严格大于均分的有效人数
    -- 避免使用 INNER JOIN + WHERE 导致无满足条件学生时该课程记录整行丢失
    SUM(CASE WHEN e.grade > v1.平均成绩 THEN 1 ELSE 0 END) AS 大于课程平均成绩的学生人数
FROM v_course_statistics v1
    JOIN courseclass cc ON v1.课程号 = cc.cno
    JOIN enrollment e ON cc.clsno = e.clsno
GROUP BY v1.课程号, v1.累计修读学生人数
[2026-04-30 10:25:47] 在 21 ms 内完成
```

#(2)查询满足下述条件的课程的课程号、课程名、开课院系：在修读过的学生中，半数以上学生的成绩低于该课程平均成绩。结果按照开课院系和课程号的升序排列。

```

SELECT
    c.cno AS 课程号,
    c.cname AS 课程名,
    c.dept AS 开课院系
FROM courses c
    JOIN v_course_statistics v1 ON c.cno = v1.课程号
    JOIN courseclass cc ON c.cno = cc.cno
    JOIN enrollment e ON cc.clsno = e.clsno
GROUP BY c.cno, c.cname, c.dept, v1.累计修读学生人数
HAVING SUM(IF(e.grade < v1.平均成绩, 1, 0)) > (v1.累计修读学生人数 / 2.0)
ORDER BY c.dept, c.cno;

```



	课程号	课程名	开课院系
1	2205	数据结构	计算机
2	1302	人工智能导论	人工智能
3	2305	机器学习	人工智能

#3.WITH CHECK OPTION 选项及视图更新

(一) 带有 WITH CHECK OPTION 选项的视图创建与查询

#(1)创建第三个视图，用于查询课程修读未通过（成绩为空或不及格）的情况，在创建视图命令中使用 WITH CHECK OPTION 选项。视图中的属性包括：学生的学号、姓名、就读院系，未通过课程的课程名、课程班代码、成绩。

```
CREATE VIEW v_failed_courses AS
```

```

SELECT
    s.sno AS 学号,
    s.sname AS 姓名,
    s.dept AS 就读院系,
    c.cname AS 课程名,
    e.clsno AS 课程班代码,
    e.grade AS 成绩
FROM enrollment e
    JOIN students s ON e.sno = s.sno
    JOIN courseclass cc ON e.clsno = cc.clsno
    JOIN courses c ON cc.cno = c.cno
WHERE e.grade IS NULL OR e.grade < 60
WITH CHECK OPTION;

```



```
[2026-04-30 10:41:51] myschool26> #3.WITH CHECK OPTION选项及视图更新
# (一) 带有WITH CHECK OPTION选项的视图创建与查询
# (1) 创建第三个视图，用于查询课程修读未通过（成绩为空或不及格）的情况，在创建视图命令中使用WITH CHECK OPTION选项。
CREATE VIEW v_failed_courses AS
SELECT
    s.sno AS 学号,
    s.sname AS 姓名,
    s.dept AS 就读院系,
    c.cname AS 课程名,
    e.clsno AS 课程班代码,
    e.grade AS 成绩
FROM enrollment e
    JOIN students s ON e.sno = s.sno
    JOIN courseclass cc ON e.clsno = cc.clsno
    JOIN courses c ON cc.cno = c.cno
WHERE e.grade IS NULL OR e.grade < 60
WITH CHECK OPTION
```

[2026-04-30 10:41:51] 在 13 ms 内完成

#(2)在第三个视图上查询课程修读未通过的情况。

SELECT * FROM v_failed_courses;

	学号	姓名	就读院系	课程名	课程班代码	成绩
1	241210032	陈玲	人工智能	高等数学(I)	2421201	41
2	211210166	蒋超	计算机	人工智能导论	2111202	55
3	231250073	王东阳	数学	数据结构	2422105	<null>
4	211210088	吴小芳	数学	数学分析	2212106	53
5	231250030	王泳桦	计算机	数据结构	2412205	57
6	221250212	陈凯歌	计算机	数据库概论	2423208	<null>

#(3)关闭事务自动提交标志: **set autocommit=0;**

set autocommit=0;

```
[2026-04-30 10:45:41] myschool26> set autocommit=0
[2026-04-30 10:45:41] 在 1 ms 内完成
```

(二) 不合要求的视图更新

#(4)选择一条成绩不及格的选课元组（记为元组t），在第三个视图上将元组t的成绩修改为60分。

UPDATE v_failed_courses

SET 成绩 = 60

WHERE 学号 = '241210032' AND 课程班代码 = '2421201';

```
[2026-04-30 10:50:32] myschool26> UPDATE v_failed_courses
    SET 成绩 = 60
    WHERE 学号 = '241210032' AND 课程班代码 = '2421201'
[2026-04-30 10:50:32] [HY000][1369] CHECK OPTION failed 'myschool26.v_failed_courses'
```

#(5)使用 **rollback** 命令放弃当前事务的更新结果，结束当前事务。

rollback;

```
[2026-04-30 10:51:48] myschool26> rollback
[2026-04-30 10:51:48] 在 1 ms 内完成
```

(三) 基表的数据更新

#(6)直接在选课表 enrollment 上, 将选课元组 t 的成绩修改为 70 分。

UPDATE enrollment

SET grade = 70

WHERE sno = '241210032' AND clsno = '2421201';

```
[2026-04-30 10:54:48] myschool26> UPDATE enrollment
                                SET grade = 70
                                WHERE sno = '241210032' AND clsno = '2421201'
[2026-04-30 10:54:48] 6 ms 中有 1 行受到影响
```

#(7)再次在第三个视图上查询课程修读未通过的情况。

SELECT * FROM v_failed_courses;

	学号	姓名	就读院系	课程名	课程班代码	成绩
1	211210166	蒋超	计算机	人工智能导论	2111202	55
2	231250073	王东阳	数学	数据结构	2422105	<null>
3	211210088	吴小芳	数学	数学分析	2212106	53
4	231250030	王泳桦	计算机	数据结构	2412205	57
5	221250212	陈凯歌	计算机	数据库概论	2423208	<null>

#(8)使用 rollback 命令放弃当前事务的更新结果, 结束当前事务。

rollback;

```
[2026-04-30 10:57:20] myschool26> rollback
[2026-04-30 10:57:20] 在 7 ms 内完成
```

(四) 符合视图定义的数据更新 (在第三个视图上完成下述数据更新)

#(9)将元组 t 的成绩修改为 50 分。

UPDATE v_failed_courses

SET 成绩 = 50

WHERE 学号 = '241210032' AND 课程班代码 = '2421201';

```
[2026-04-30 11:00:42] myschool26> UPDATE v_failed_courses
                                SET 成绩 = 50
                                WHERE 学号 = '241210032' AND 课程班代码 = '2421201'
[2026-04-30 11:00:42] 2 ms 中有 1 行受到影响
```

#(10)将‘数据结构’课程的不及格成绩统一修改为空值。

UPDATE v_failed_courses

SET 成绩 = NULL

WHERE 课程名 = '数据结构'

AND 成绩 < 60;

```
[2026-04-30 11:02:48] myschool26> UPDATE v_failed_courses
                                SET 成绩 = NULL
                                WHERE 课程名 = '数据结构'
                                AND 成绩 < 60
[2026-04-30 11:02:48] 4 ms 中有 1 行受到影响
```

#(11)最后, 再一次在第三个视图上查询课程修读未通过的情况。

SELECT * FROM v_failed_courses;

	学号	姓名	就读院系	课程名	课程班代码	成绩
1	241210032	陈玲	人工智能	高等数学(1)	2421201	50
2	211210166	蒋超	计算机	人工智能导论	2111202	55
3	231250073	王东阳	数学	数据结构	2422105	<null>
4	211210088	吴小芳	数学	数学分析	2212106	53
5	231250030	王泳桦	计算机	数据结构	2412205	<null>
6	221250212	陈凯歌	计算机	数据库概论	2423208	<null>

#(12)使用 rollback 命令放弃当前事务的更新结果，结束当前事务。

rollback;

```
[2026-04-30 11:04:34] myschool26> rollback
[2026-04-30 11:04:34] 在 5 ms 内完成
```

#4.删除在本实验创建的所有视图。

DROP VIEW IF EXISTS v_course_performance;

DROP VIEW IF EXISTS v_failed_courses;

DROP VIEW IF EXISTS v_course_statistics;

```
[2026-04-30 11:06:35] myschool26> DROP VIEW IF EXISTS v_course_performance
[2026-04-30 11:06:35] 在 15 ms 内完成
[2026-04-30 11:06:35] myschool26> DROP VIEW IF EXISTS v_failed_courses
[2026-04-30 11:06:35] 在 10 ms 内完成
[2026-04-30 11:06:35] myschool26> DROP VIEW IF EXISTS v_course_statistics
[2026-04-30 11:06:35] 在 9 ms 内完成
```

实验中遇到的困难及解决办法

[详细说明你认为本次实验中比较困难的地方，也可以对实验设计提出建议]

第一次写数据更新、视图创建与使用相关的 SQL 语句，较为生疏

参考文献及致谢

[如果你参考了任何书籍、网页，或与他人进行了讨论，请在此注明]

解决第七题时 **SELECT COUNT(DISTINCT cc.cno)** 这里一开始没有写 **distinct**，后面与同学讨论时发现应该加上 **distinct** 去重来防止存在一个学生重复修本院某一门课程多次且均及格的情况（尽管样例中没有）